
1. What Is the Task Class?

In C#, **Task** represents an **asynchronous operation**.

It is part of the **TPL (Task Parallel Library)** introduced in .NET 4.0.

Key points:

- A Task **does not start automatically** unless created via `Task.Run` or `Task.Factory.StartNew`.
- It represents a **unit of work** that runs **asynchronously** (usually on a thread pool thread).
- It **may or may not return a value**.
- It integrates with the **async / await** programming model.
- It provides **state, lifecycle, cancellation, continuations, and exception handling**.

2. Two Types of Tasks

Type	Definition	Example
Task	Performs work without returning a value	Saving a log, sending an email
Task<TResult>	Performs work and returns a result	Fetching data, calculating a number

Example:

```
Task task = DoSomethingAsync();
```

```
Task<int> task2 = CalculateValueAsync();
```

3. Why Do We Use Task?

Because we want to:

1. **Avoid blocking the main thread**
Example: Do not freeze the UI during download.
2. **Improve performance**
Run I/O operations in parallel.

3. Write scalable APIs

ASP.NET Core APIs use async to handle more concurrent requests.

4. Use async/await easily

Task integrates naturally with C# async model.

4. Task Lifecycle

A Task goes through several states:

Created → WaitingToRun → Running → RanToCompletion

 ↳ Faulted

 ↳ Canceled

You can check using:

task.Status

task.IsCompleted

task.IsFaulted

task.IsCanceled

5. How to Create and Run Tasks

5.1 Using Task.Run() (most common)

Task task = Task.Run(() =>

```
{  
    Console.WriteLine("Task running...");  
});
```

5.2 Using async Method Returning a Task

This is the recommended way:

async Task ProcessAsync()

```
{  
    Console.WriteLine("Processing...");
```

```
    await Task.Delay(1000);  
    Console.WriteLine("Done");  
}
```

5.3 Using a Task with Return Value

```
async Task<int> AddAsync(int a, int b)  
{  
    await Task.Delay(500);  
    return a + b;  
}
```

Calling it:

```
int sum = await AddAsync(5, 7);  
Console.WriteLine(sum);
```

6. Task vs Thread (Conceptual Difference)

Thread	Task
Physical worker	Logical representation of work
OS resource	Lightweight
Expensive	Efficient
Manual lifecycle	Managed by Task Scheduler
No async/await integration	Native async support

7. Task Examples (Hands-On)

Example 1: Simple Task Without Return

```
static async Task Demo1()  
{
```

```
Console.WriteLine("Starting...");
```

```
Task t = Task.Run(() =>
```

```
{
```

```
    Thread.Sleep(3000);
```

```
    Console.WriteLine("Task completed");
```

```
});
```

```
await t; // Wait for task but without blocking thread
```

```
Console.WriteLine("After Task");
```

```
}
```

Output:

Starting...

Task completed

After Task

Example 2: Task With Return Value

```
static async Task Demo2()
```

```
{
```

```
    Task<int> t = Task.Run(() =>
```

```
{
```

```
    Thread.Sleep(2000);
```

```
    return 10 * 10;
```

```
});
```

```
int result = await t;

Console.WriteLine($"Result: {result}");
}
```

Example 3: Task.Delay (Non-blocking Wait)

Task.Delay does NOT block the thread.
It returns a task that completes after a time period.

```
await Task.Delay(3000);

Console.WriteLine("3 seconds passed");
```

Example 4: Multiple Tasks Running in Parallel

```
Task t1 = Task.Run(() => Console.WriteLine("Task 1"));
Task t2 = Task.Run(() => Console.WriteLine("Task 2"));

await Task.WhenAll(t1, t2);
```

Example 5: Error Handling in Tasks

```
try
{
    await Task.Run(() =>
    {
        throw new Exception("Something failed...");
    });
}
catch (Exception ex)
```

```
{  
    Console.WriteLine(ex.Message);  
}
```

8. Continuation Tasks

Tasks can chain operations.

```
Task.Run(() => Console.WriteLine("Step 1"))  
    .ContinueWith(t => Console.WriteLine("Step 2"))  
    .ContinueWith(t => Console.WriteLine("Step 3"));
```

However, with `async/await`, continuation tasks are rarely used anymore.

9. Task.WhenAll & Task.WhenAny

WhenAll (Wait for all tasks)

```
await Task.WhenAll(Task1(), Task2(), Task3());
```

WhenAny (Wait for first completed)

```
Task first = await Task.WhenAny(Task1(), Task2());
```

10. Task Cancellation

Use `CancellationToken`.

```
var cts = new CancellationTokenSource();
```

```
CancellationToken token = cts.Token;
```

```
Task t = Task.Run(() =>  
{  
    for (int i = 0; i < 5; i++)  
    {
```

```
token.ThrowIfCancellationRequested();  
Thread.Sleep(1000);  
Console.WriteLine("Working...");  
}  
, token);
```

```
cts.Cancel(); // Try to cancel the task
```

```
await t;
```

Summary (Conceptual)

1. **Task** = asynchronous unit of work.
 2. Two types: Task (void) and Task<TResult> (returns value).
 3. Use async/await to simplify asynchronous code.
 4. Tasks run on the **ThreadPool**, not UI thread.
 5. Tasks support:
 - Cancellation
 - Exception handling
 - Continuations
 - Parallel execution
 6. Recommended way to write async methods:
 7. `async Task MethodName() { ... }`
-